

Allineamento Progressivo nel T-Coffee

Ascolese - D'Andrea
Anno Accademico 2025 - 2026

Indice

1	Allineamento Multiplo: modelli di scoring, programmazione dinamica ed euristiche	3
1.1	Modelli di scoring per allineamenti proteici	4
1.1.1	Interpretazione probabilistica dello scoring	4
1.2	Penalità di gap e modellazione degli indel	5
1.3	Programmazione dinamica come nucleo computazionale	5
1.4	Allineamento multiplo: complessità ed euristiche	6
1.5	Qualità del dataset e robustezza dell'MSA	6
2	Allineamento Progressivo	7
2.1	Fasi dell'algoritmo	7
2.2	Allineamento Progressivo in Python	8
2.3	Profili e scoring	10
2.4	Limiti dell'approccio	10
2.5	Evoluzioni e ottimizzazioni	10
3	T-Coffee	12
3.1	Consistenza Biologica	13
3.1.1	Funzione obiettivo basata sulla consistenza (COF-FEE score)	13
3.2	L'Algoritmo T-Coffee	15
3.2.1	Generazione della Libreria Primaria	15
3.2.2	Estensione della Libreria	16
3.2.3	Allineamento Progressivo	17
3.2.4	Gestione delle penalità di gap	18
3.3	Confronto tra T-Coffee e allineamento progressivo classico (Clustal)	19
3.4	Analisi della Complessità Computazionale	20
3.5	Vantaggi e Limitazioni	21
3.6	Integrazione di allineamenti locali e globali nella libreria	22
3.7	Valutazione dell'affidabilità: indice CORE	22
3.8	Conclusioni sul Metodo	23
3.9	Integrazione di Allineamenti Locali e Globali nella Libreria	23

3.10	Evoluzione della Piattaforma T-Coffee: dal 2000 al 2025	24
3.10.1	L'Era dell'Accuratezza e dell'Integrazione (2000-2010)	24
3.10.2	L'Era della Scalabilità e dell'Integrazione Strutturale Predittiva (2015-2025)	25
A	Implementazione: scheletro Python per T-Coffee	27
A.1	Struttura generale	27
	Bibliografia	38

1 Allineamento Multiplo: modelli di scoring, programmazione dinamica ed euristiche

L'**allineamento multiplo di sequenze** (*Multiple Sequence Alignment*, MSA) è uno dei problemi centrali della *sequence analysis*, perché consente di confrontare simultaneamente più sequenze (proteiche o nucleotidiche) per evidenziare **posizioni conservate**, **motivi funzionali**, **domini condivisi** e **relazioni evolutive**. Rispetto a un allineamento pairwise, l'MSA aumenta l'informazione disponibile: ambiguità che in un confronto a due sequenze resterebbero irrisolte possono essere vincolate dall'evidenza fornita da sequenze aggiuntive. In pratica, un MSA ben costruito diventa una rappresentazione compatta di un'intera famiglia, utile per annotazione funzionale, analisi filogenetiche e supporto a inferenze strutturali.

Dal punto di vista algoritmico, un allineamento va interpretato come **ipotesi operativa** sulle corrispondenze tra residui omologhi. Tale ipotesi viene formalizzata tramite una funzione obiettivo: si definisce uno **schema di scoring** e si ricerca l'allineamento (o la classe di allineamenti) che massimizza tale punteggio. Di conseguenza, la qualità dell'MSA dipende in modo cruciale da due dimensioni: (i) **modello di scoring** (matrici di sostituzione, penalità di gap, parametri); (ii) **strategia di costruzione dell'allineamento** (euristica/procedura: progressive, iterativa, consistency-based, ecc.). Il focus metodologico non è quindi "allineare e basta", ma progettare una pipeline in cui **scelte di scoring** e **scelte algoritmiche** siano coerenti con il problema biologico (omologhi vicini vs distanti, sequenze multidominio, regioni a bassa complessità, ecc.).

1.1 Modelli di scoring per allineamenti proteici

La percent identity è una misura intuitiva ma concettualmente limitata, poiché valuta le corrispondenze in modo binario (identico/non identico) e non distingue tra sostituzioni conservative e sostituzioni improbabili dal punto di vista evolutivo. Per questo motivo, negli allineamenti proteici si utilizzano matrici di sostituzione che assegnano a ogni coppia di residui un punteggio proporzionale alla probabilità osservata di sostituzione in posizioni omologhe, rispetto al caso.

1.1.1 Interpretazione probabilistica dello scoring

Dal punto di vista probabilistico, la compatibilità di una coppia di residui (a, b) con un modello evolutivo può essere espressa tramite un *odds ratio*:

$$\text{odds}(a, b) = \frac{q_{a,b}}{p_a p_b},$$

dove $q_{a,b}$ è la probabilità di osservare la coppia (a, b) in posizioni omologhe, mentre p_a e p_b sono le frequenze di background dei residui.

Le matrici di sostituzione (PAM, BLOSUM) sono costruite come matrici di *log-odds score*:

$$s(a, b) = \log \frac{q_{a,b}}{p_a p_b}.$$

Valori positivi indicano sostituzioni più probabili sotto il modello evolutivo che sotto il caso, mentre valori negativi indicano corrispondenze sfavorite. Questa interpretazione fornisce una base formale allo scoring degli allineamenti e consente di sommare i contributi delle singole posizioni lungo l'allineamento.

Dato un allineamento pairwise A , il suo punteggio complessivo può essere espresso come:

$$S(A) = \sum_u s(a_u, b_u),$$

dove u scorre sulle posizioni allineate. Nei metodi di allineamento multiplo, questo principio viene esteso integrando i contributi su tutte le coppie di sequenze e su tutte le colonne (criterio *sum-of-pairs*).

1.2 Penalità di gap e modellazione degli indel

Sequenze omologhe possono differire in lunghezza a causa di eventi di inserzione e delezione (indel). Per rappresentare tali eventi, l'allineamento introduce gap, la cui gestione è regolata da penalità sottratte allo score complessivo. Il modello più utilizzato è la penalità affine:

$$g(n) = -I - (n - 1)E,$$

dove I rappresenta il costo di apertura del gap ed E il costo di estensione.

Questo schema riflette una considerazione biologica essenziale: gli indel tendono a manifestarsi come eventi estesi piuttosto che come gap isolati. Dal punto di vista dell'MSA, la scelta delle penalità di gap influisce direttamente sulla stabilità delle colonne, sulla crescita dei profili durante l'allineamento progressivo e sulla corretta separazione tra regioni core e regioni variabili.

1.3 Programmazione dinamica come nucleo computazionale

Per due sequenze, la ricerca dell'allineamento ottimo rispetto a uno schema di scoring definito può essere effettuata esattamente tramite programmazione dinamica (DP). Algoritmi classici come Needleman–Wunsch (allineamento globale) e Smith–Waterman (allineamento locale) rendono trattabile uno spazio di soluzioni altrimenti esponenziale, sfruttando il principio di ottimalità: un allineamento ottimo globale non può contenere sottoparti non ottime rispetto allo stesso schema di scoring.

Sebbene l'estensione diretta della DP a più di due sequenze sia computazionalmente impraticabile, la DP rimane il nucleo di ottimizzazione locale anche negli MSA moderni. In particolare, i metodi multipli riducono il problema a una sequenza di allineamenti sequenza–profilo e profilo–profilo, che sono formalmente estensioni della DP pairwise, con funzioni di scoring definite su colonne e obiettivi aggregati.

1.4 Allineamento multiplo: complessità ed euristiche

L'allineamento simultaneo di n sequenze richiederebbe una programmazione dinamica n -dimensionale, con costo computazionale proibitivo. Per questo motivo, gli MSA vengono costruiti quasi esclusivamente tramite euristiche. Il paradigma più diffuso è l'allineamento progressivo, adottato da strumenti come ClustalW, che procede attraverso: (i) il calcolo delle similarità pairwise, (ii) la costruzione di un albero guida, e (iii) l'allineamento progressivo delle sequenze e dei profili seguendo la struttura dell'albero.

Il limite fondamentale di questo approccio è che le decisioni prese nelle fasi iniziali non vengono corrette: errori precoci possono propagarsi e vincolare negativamente l'allineamento finale. Questa dipendenza dalla guida iniziale motiva lo sviluppo di strategie più avanzate, come i metodi iterativi e *consistency-based*, che cercano di integrare evidenza multipla e ridurre la sensibilità a scelte locali sub-ottimali.

1.5 Qualità del dataset e robustezza dell'MSA

La qualità dell'MSA dipende anche dalla composizione del dataset. Regioni a bassa complessità o con forte bias composizionale possono produrre score elevati in assenza di reale omologia, introducendo rumore e instabilità nell'allineamento. Per questo motivo, nelle pipeline di ricerca e di costruzione dell'MSA vengono comunemente applicati filtri di mascheramento, al fine di migliorare l'affidabilità dell'evidenza utilizzata nelle fasi successive.

2 Allineamento Progressivo

L'allineamento progressivo è una delle principali euristiche per la costruzione di un allineamento multiplo di sequenze (Multiple Sequence Alignment, MSA). In un MSA, l'obiettivo è allineare simultaneamente un insieme di sequenze biologiche per evidenziare regioni conservate, motivi funzionali e relazioni evolutive. Il problema di ottenere l'allineamento multiplo ottimo tramite programmazione dinamica multidimensionale ha complessità esponenziale $\mathcal{O}(L^N)$, dove L è la lunghezza delle sequenze e N il loro numero, rendendo impraticabile un approccio esatto per dataset di dimensioni realistiche.

L'allineamento progressivo affronta questa difficoltà con una strategia di tipo greedy, orientata all'efficienza, costruendo l'allineamento tramite passi incrementali. L'idea di base è di allineare prima le sequenze più simili e di integrare progressivamente le sequenze più distanti, riducendo la complessità computazionale complessiva a un ordine di grandezza gestibile per dataset di dimensioni medie.

2.1 Fasi dell'algoritmo

L'allineamento progressivo si articola in tre fasi principali:

Calcolo della matrice delle distanze. Si calcolano confronti pairwise tra tutte le coppie di sequenze per ottenere una matrice di distanze evolutive. I punteggi di similarità o distanza possono essere ottenuti da allineamenti globali o metriche approssimate basate su conteggi di *k-tuple*. La matrice risultante sintetizza le relazioni di similarità tra tutte le sequenze.

Costruzione dell'albero guida. A partire dalla matrice delle distanze si costruisce un albero guida (guide tree) tramite algoritmi di clustering gerarchico come UPGMA o Neighbor-Joining. L'albero non rappresenta una ricostruzione filogenetica rigorosa, ma stabilisce l'ordine in cui sequenze (o gruppi di sequenze) verranno allineate.

Allineamento incrementale. Seguendo l'albero guida dalle foglie verso la radice, si allineano inizialmente le coppie più simili; successivamente, le sequenze vengono integrate allineandole a profili già costruiti. In questa fase si realizzano allineamenti *sequence-profile* e *profile-profile*, per i quali si utilizzano funzioni di scoring derivate dalla somma dei punteggi di coppia (*sum-of-pairs*). L'implementazione di base di questa logica è illustrata nel Codice 1 (Appendice), in cui due profili vengono allineati tramite una matrice di programmazione dinamica custom che utilizza uno scoring normalizzato tra colonne.

2.2 Allineamento Progressivo in Python

Scrivere un MSA completo ,come Clustal, richiede migliaia di righe, ma il concetto chiave che differenzia l'allineamento progressivo da quello a coppie è la capacità di allineare un profilo contro un altro profilo. In questo codice, è simulato l'ultimo passaggio: ci sono due gruppi di sequenze già pre-allineate (Profili) ed il codice va ad unirle. Dal punto di vista algoritmico, questo tipo di allineamento profilo-profilo può essere implementato come una programmazione dinamica su colonne, estendendo lo schema classico di Needleman-Wunsch a funzioni di scoring tra insiemi di residui.

```
1 import numpy as np
2
3 def get_profile_column_score(col_a, col_b):
4     """
5     Calcola il punteggio tra due colonne di due profili diversi.
6     Usa la logica 'Sum of Pairs'.
7     """
8     score = 0
9     # Confronta ogni residuo del primo profilo con ogni residuo del
10    secondo
11    for char_a in col_a:
12        for char_b in col_b:
13            if char_a == '-' and char_b == '-':
14                step_score = 0 # Gap vs Gap (spesso ignorato)
15            elif char_a == '-' or char_b == '-':
16                step_score = -2 # Penalita' gap
17            elif char_a == char_b:
18                step_score = 1 # Match
19            else:
```

```

19         step_score = -1 # Mismatch
20
21         score += step_score
22
23         # Normalizzazione per evitare che profili grandi dominino il
24         # punteggio
25         return score / (len(col_a) * len(col_b))
26
27 def align_profiles(profile_A, profile_B):
28     """
29     Allinea due profili (liste di stringhe gia' allineate
30     internamente).
31     Input: profile_A, profile_B (liste di stringhe)
32     Output: Score dell'allineamento ottimale
33     """
34
35     n = len(profile_A[0]) # Lunghezza profilo A
36     m = len(profile_B[0]) # Lunghezza profilo B
37
38     # Inizializzazione matrice DP
39     score_matrix = np.zeros((n + 1, m + 1))
40
41     # Inizializzazione prima riga e colonna (Gap penalties lineari)
42     for i in range(1, n + 1):
43         score_matrix[i][0] = score_matrix[i-1][0] - 2
44     for j in range(1, m + 1):
45         score_matrix[0][j] = score_matrix[0][j-1] - 2
46
47     # Riempimento matrice (Fase Forward)
48     for i in range(1, n + 1):
49         for j in range(1, m + 1):
50             # Estraiamo le colonne i-esima e j-esima dai profili
51             col_a = [seq[i-1] for seq in profile_A]
52             col_b = [seq[j-1] for seq in profile_B]
53
54             match_score = get_profile_column_score(col_a, col_b)
55
56             # Calcolo dei punteggi per le tre direzioni possibili
57             score_diag = score_matrix[i-1][j-1] + match_score
58             score_up = score_matrix[i-1][j] - 2 # Gap in B
59             score_left = score_matrix[i][j-1] - 2 # Gap in A
60
61             # Scelta del massimo (approccio Greedy/DP)
62             score_matrix[i][j] = max(score_diag, score_up, score_left)
63
64     return score_matrix[n][m]

```

Listing 1: Implementazione dell'allineamento progressivo tra due profili

2.3 Profili e scoring

Un profilo rappresenta un insieme di sequenze già allineate, descritto in termini di colonne contenenti residui o gap. Il punteggio tra due colonne di profili viene calcolato sommando (o mediando) i contributi di tutte le coppie di residui appartenenti alle due colonne, eventualmente includendo penalità per gap. La funzione `get_profile_column_score` nel Codice 1 implementa questa metrica di *sum-of-pairs* per colonne di profilo.

In implementazioni reali, i profili possono essere arricchiti da matrici di punteggio posizione-specifiche (PSSM) e stabilizzati tramite pseudocounts e pesi di sequenza. Questi miglioramenti aumentano la robustezza delle stime di frequenza residue quando il numero di sequenze è ridotto, ma non sono essenziali per comprendere il funzionamento euristico di base del progressive alignment.

2.4 Limiti dell'approccio

Il principale limite dell'allineamento progressivo deriva dalla sua natura greedily: i gap introdotti nelle fasi iniziali non vengono modificati successivamente (“*once a gap, always a gap*”), e quindi errori nell'allineamento precoce possono propagarsi all'intero MSA. In termini algoritmici, l'euristica non ottimizza una funzione obiettivo globale, ma una sequenza di obiettivi locali condizionati alle decisioni già prese. Questo comportamento rende l'allineamento progressivo sensibile alla qualità dell'albero guida e delle metriche di distanza iniziali.

2.5 Evoluzioni e ottimizzazioni

Per mitigare tali limiti, sono stati sviluppati approcci più sofisticati. I metodi di *iterative refinement* (ad es. MAFFT, MUSCLE) introducono fasi di revisione in cui sottoinsiemi di sequenze vengono temporaneamente rimossi e riallineati per migliorare lo score globale. Un'altra classe di strategie è rappresentata dai metodi *consistency-based*, che integrano informazioni da allineamenti locali e globali preliminari come

vincoli addizionali durante la costruzione progressiva dell'MSA; questa idea è alla base di strumenti come T-Coffee, che verrà trattato nel capitolo successivo.

Nonostante i suoi limiti, l'allineamento progressivo rimane la pietra angolare di molte pipeline MSA grazie al suo equilibrio tra accuratezza ed efficienza computazionale.

3 T-Coffee

Sebbene l'allineamento progressivo classico, esemplificato dalla famiglia di software Clustal, rappresenti un'euristica efficace per ridurre la complessità computazionale dell'allineamento multiplo (MSA), esso soffre intrinsecamente della natura "greedy" (ingorda) dell'algoritmo.

La limitazione principale risiede nell'impossibilità di correggere gli errori commessi nelle prime fasi della costruzione dell'allineamento: una volta inserito un gap, questo viene propagato irreversibilmente.

Per mitigare questo problema senza abbandonare l'efficienza dello schema progressivo, nel 2000 Notredame, Higgins e Heringa hanno introdotto T-Coffee (Tree-based Consistency Objective Function For alignmEnt Evaluation). T-Coffee non abbandona l'architettura progressiva, ma modifica in modo sostanziale il modo in cui vengono valutate le corrispondenze: l'informazione proveniente da allineamenti pairwise (calcolati con scoring standard, tipicamente basato su matrici di sostituzione e penalità di gap) viene prima raccolta in una *libreria* di vincoli pesati. Nella fase progressiva finale, la programmazione dinamica non utilizza più direttamente una matrice di sostituzione statica per assegnare il punteggio colonna-colonna, ma uno score derivato dal supporto fornito dalla libreria estesa secondo il principio di consistenza.

Al fine di chiarire il funzionamento dell'algoritmo T-Coffee, introduciamo un *esempio guida*, ispirato a quello presentato nel lavoro originale di Notredame et al. (2000). L'obiettivo è seguire lo stesso insieme di sequenze lungo tutte le fasi dell'algoritmo, evidenziando come l'informazione venga progressivamente integrata e raffinata secondo una visione top-down.

Si considerino tra sequenze proteiche S_1 , S_2 , S_3 che condividono un' omologia globale ma presentano regioni localmente divergenti. In un allineamento progressivo classico, eventuali errori introdotti nelle prime fasi (ad esempio un posizionamento scorretto dei gap) tendono a propagarsi irreversibilmente. T-Coffee affronta questo problema separando concettualmente la raccolta dell'evidenza di omologia dalla costruzione dell'allineamento multiplo finale.

Nel seguito del capitolo, questo esempio verrà ripreso in ciascuna

fase dell'algoritmo (generazione della libreria primaria, estensione per consistenza e allineamento progressivo), così da fornire una lettura coerente e unificata del metodo.

3.1 Consistenza Biologica

L'idea fondamentale di T-Coffee è che l'allineamento tra due sequenze, S_A e S_B , non debba dipendere esclusivamente dal confronto diretto tra i loro residui, ma debba essere informato dal modo in cui entrambe si allineano con tutte le altre sequenze del set. Se il residuo x della sequenza A è allineato con il residuo z della sequenza C , e lo stesso residuo z è allineato con il residuo y della sequenza B , allora esiste una "evidenza transitoria" che x e y debbano essere allineati tra loro, anche se il confronto diretto $A - B$ (pairwise) non lo suggerisce fortemente. Questa proprietà transitiva è definita consistenza.

3.1.1 Funzione obiettivo basata sulla consistenza (COFFEE score)

Il principio di consistenza non è soltanto un'euristica locale, ma induce una funzione obiettivo globale per l'MSA. Sia $\mathcal{L}$ la libreria (estesa) di vincoli residuo-residuo, dove ogni vincolo collega due residui (i, a) e (j, b) ed è associato a un peso $w_{i,a,j,b}^{\text{ext}} \geq 0$. Un allineamento multiplo finale A induce un insieme di coppie residuo-residuo poste nella stessa colonna (escludendo i gap). Il punteggio complessivo dell'allineamento può allora essere espresso come la somma dei pesi dei vincoli della libreria che risultano *soddisfatti* da A :

$$\text{Score}(A) = \sum_{(i,a,j,b) \in A} w_{i,a,j,b}^{\text{ext}}.$$

In questo senso, T-Coffee non massimizza direttamente una similarità locale basata su una matrice di sostituzione, ma ricerca un allineamento che massimizzi la coerenza globale con l'evidenza integrata nella libreria (ottenuta da più confronti pairwise e, in particolare, da segnali locali e globali).

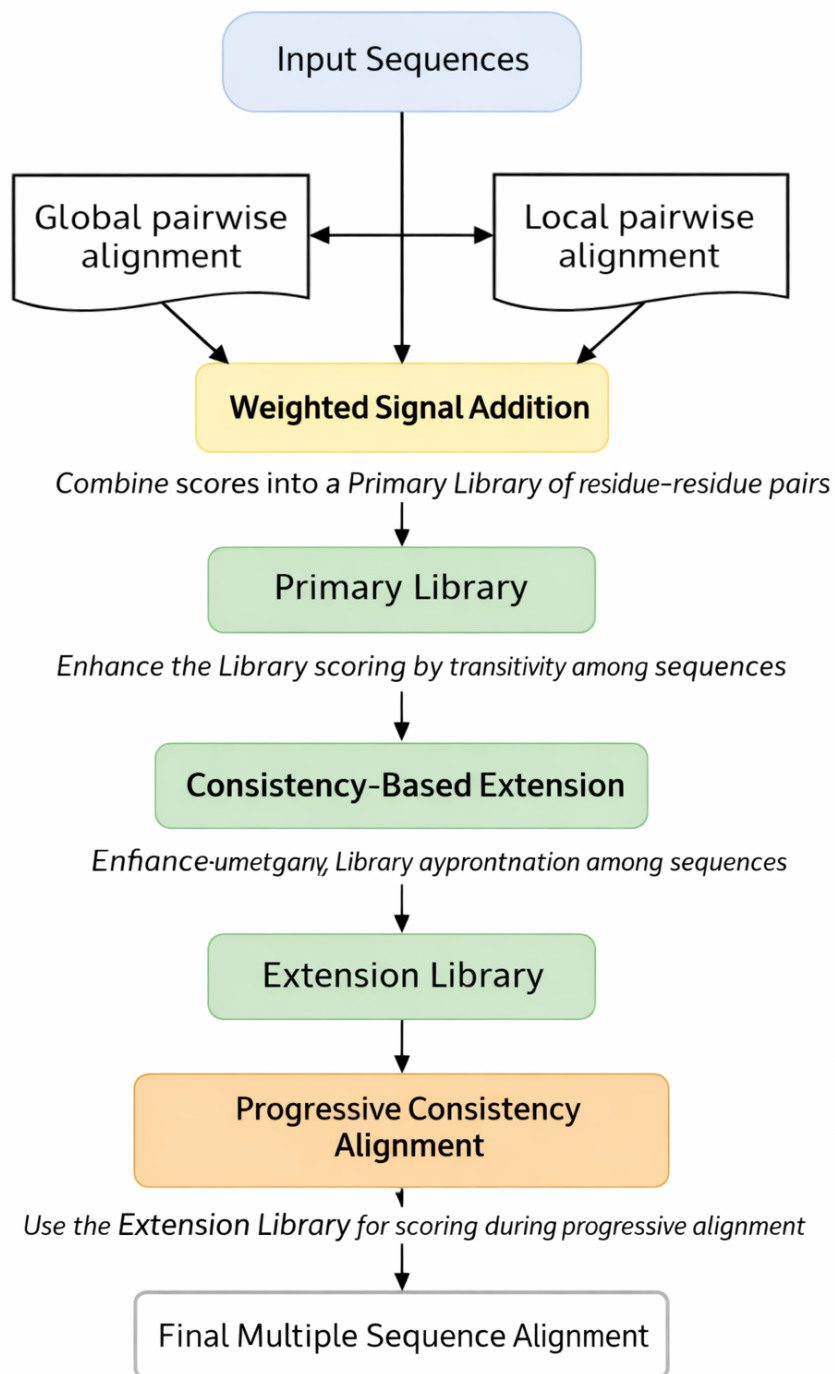


Figura 1: Pipeline concettuale dell'algoritmo T-Coffee.

3.2 L'Algoritmo T-Coffee

Il flusso di lavoro di T-Coffee si articola in tre fasi distinte, che precedono e modificano la classica procedura di allineamento progressivo.

3.2.1 Generazione della Libreria Primaria

Riprendendo l'esempio guida introdotto in precedenza, T-Coffee calcola gli allineamenti pairwise tra tutte le coppie di sequenze (S_1-S_2 , S_1-S_3 , S_2-S_3), estraendo da ciascuno le coppie di residui allineate che costituiranno la libreria primaria di vincoli. Invece di calcolare una semplice matrice di distanze, T-Coffee genera una libreria di allineamenti. Per ogni coppia di sequenze, vengono calcolati due tipi di allineamento:

- Allineamento Globale: Solitamente ottenuto tramite l'algoritmo ClustalW (Needleman-Wunsch).
- Allineamento Locale: Ottenuto tramite l'algoritmo LALIGN (Smith-Waterman), per identificare regioni conservate o domini condivisi anche in presenza di scarsa similarità globale.

Ogni coppia di residui allineati (x, y) proveniente da questi allineamenti viene inserita nella libreria con un peso iniziale pari alla percentuale di identità dell'allineamento da cui proviene.

Rappresentazione della libreria La libreria primaria può essere vista come una collezione di vincoli *residuo-residuo* pesati, ciascuno dei quali collega una posizione specifica di una sequenza a una posizione specifica di un'altra. In termini implementativi (o, più astrattamente, di strutture dati), è naturale rappresentarla come una mappa/lookup table indicizzata da una quadrupla che identifica univocamente i due residui:

$$(i, a, j, b) \quad \text{con } i < j$$

dove i e j sono gli identificatori delle sequenze e a, b le posizioni dei residui (nelle sequenze originali, prima dell'MSA). Il valore associato è un peso $w_{i,a,j,b}$ che quantifica la confidenza del vincolo. Quando più

sorgenti contribuiscono allo stesso vincolo (ad esempio un allineamento globale e uno locale per la stessa coppia di sequenze), i contributi vengono combinati (ad esempio per somma o media pesata) $w_{i,a,j,b} \leftarrow w_{i,a,j,b} + w_{\text{sorgente}}$. Questa rappresentazione rende immediato, nelle fasi successive, recuperare il supporto di consistenza per ogni potenziale allineamento tra residui

3.2.2 Estensione della Libreria

Nel contesto dell'esempio guida, l'estensione della libreria consente di rafforzare un vincolo tra due residui di S_1 e S_3 qualora entrambi risultino coerentemente allineati a un residuo intermedio di S_2 , anche in assenza di un forte supporto diretto nel confronto pairwise. La fase di *library extension* è l'elemento che distingue T-Coffee dai metodi progressivi classici. L'idea è rafforzare (o indebolire) il peso di un vincolo tra due residui non solo in base al loro allineamento diretto, ma anche in base alla coerenza con gli allineamenti che coinvolgono le altre sequenze del dataset. Siano (i, a) un residuo della sequenza S_i e (j, b) un residuo della sequenza S_j . Il vincolo diretto tra questi due residui, rappresentato dal peso $w_{i,a,j,b}$, può essere supportato indirettamente da una terza sequenza S_k se esiste una posizione c tale che i vincoli $(i, a) \leftrightarrow (k, c)$ e $(k, c) \leftrightarrow (j, b)$ siano entrambi presenti e forti. In tal caso, la confidenzialità nell'associazione $(i, a) \leftrightarrow (j, b)$ aumenta, perchè l'evidenza è consistente lungo il percorso $S_i \rightarrow S_k \rightarrow S_j$.

Operativamente, l'estensione ricalcola il peso come somma del contributo diretto e di contributi indiretti mediati da tutte le sequenze intermedie:

$$w'_{i,a,j,b} = w_{i,a,j,b} + \sum_{k \neq i,j} \sum_c f(w_{i,a,k,c}, w_{k,c,j,b}),$$

dove $f(\cdot, \cdot)$ è una funzione di combinazione che modella il principio del *collo di bottiglia*: supporto indiretto non può essere più forte del legame più debole lungo la catena. Una scelta comune è

$$f(x, y) = \min(x, y),$$

oppure, in varianti equivalenti, una forma moltiplicativa o normalizzata. Questo meccanismo amplifica i vincoli compatibili attraverso molte sequenze e attenua i vincoli isolati, riducendo la propagazione di errori tipica degli approcci progressivi puramente greedy.

3.2.3 Allineamento Progressivo

Una volta costruita la libreria estesa, l'allineamento progressivo delle sequenze dell'esempio guida procede seguendo l'albero guida, ma utilizzando come funzione di scoring il supporto fornito dai vincoli di consistenza, anziché una matrice di sostituzione statica. Una volta ottenuta la libreria estesa, T-Coffee procede con lo schema progressivo standard:

1. Calcolo della matrice delle distanze basata sui pesi della libreria.
2. Costruzione dell'albero guida (Neighbor-Joining).
3. Allineamento progressivo seguendo l'albero.

La differenza sostanziale rispetto a Clustal è che, durante l'allineamento di due profili, il punteggio per allineare una colonna del profilo P_1 con una colonna del profilo P_2 non è dato da una matrice di sostituzione statica (ad esempio BLOSUM), ma da una funzione di scoring basata sulla libreria estesa di consistenza.

Siano C_1 e C_2 due colonne appartenenti rispettivamente ai profili P_1 e P_2 . Il punteggio associato alla mossa diagonale della programmazione dinamica è definito come:

$$\text{Score}(C_1, C_2) = \sum_{\substack{(i,a) \in C_1 \\ (j,b) \in C_2}} w_{i,a,j,b}^{\text{ext}},$$

dove la sommatoria considerata tutte le coppie di residui reali (escludendo i gap), identificati dalla coppia sequenza, posizione originale e $w_{i,a,j,b}^{\text{ext}}$ rappresenta il peso del vincolo residuo-residuo nella libreria estesa.

Ricostruzione dell'allineamento (traceback). La programmazione dinamica utilizzata per l'allineamento di sequenze o profili restituisce uno *score ottimo*, ma per ottenere l'allineamento esplicito è necessario memorizzare anche le scelte che conducono a tale ottimo. A questo scopo, oltre alla matrice degli score, si mantiene una matrice di *traceback* che registra, per ogni cella, quale transizione (diagonale, verticale, orizzontale) ha prodotto il valore massimo.

Terminato il riempimento delle matrici, l'allineamento viene ricostruito percorrendo a ritroso il traceback dalla cella finale all'origine. Nel caso di un allineamento *profilo-profilo*, tale procedura consente di costruire un nuovo profilo risultante: le colonne dei due profili vengono emesse in uscita seguendo il percorso di traceback, introducendo gap nelle righe appropriate e mantenendo la corrispondenza con le posizioni originali. Questo passaggio è essenziale perché T-Coffee produce un MSA completo, ottenuto iterando l'allineamento e l'aggiornamento dei profili lungo l'albero guida.

Costruzione dell'albero guida Come nei metodi progressivi classici, T-Coffee utilizza un albero guida (*guide tree*) per determinare l'ordine con cui sequenze e profili vengono progressivamente allineati. L'albero è costruito a partire da una matrice di distanze derivate dagli allineamenti pairwise iniziali, utilizzando algoritmi di clustering gerarchico quali UPGMA o Neighbor-Joining. Le sequenze più simili vengono allineate per prime, generando profili intermedi che vengono successivamente combinati lungo la topologia dell'albero.

È importante sottolineare che l'albero guida non rappresenta l'innovazione di T-Coffee: la struttura progressiva rimane invariata rispetto a Clustal. La differenza fondamentale risiede nella funzione di scoring utilizzata durante l'allineamento, che in T-Coffee è guidata dalla libreria di consistenza piuttosto che da una matrice di sostituzione statica.

3.2.4 Gestione delle penalità di gap

Nelle implementazioni reali di T-Coffee, la gestione dei gap è più sofisticata rispetto ai modelli didattici basati su una penalità lineare

costante. In particolare, vengono comunemente utilizzate penalità di gap di tipo affine, che distinguono tra il costo di apertura di un gap e il costo di estensione dello stesso.

Dal punto di vista algoritmico, ciò si traduce nell'uso di tre matrici di programmazione dinamica: una matrice M per gli allineamenti residuo-residuo, e due matrici I_x e I_y per rappresentare gap aperti e prolungati rispettivamente in una delle due sequenze (o profili). Questo schema consente di modellare in modo più realistico eventi biologici come inserzioni e delezioni contigue.

Inoltre, T-Coffee può modulare la penalizzazione dei gap in funzione del supporto fornito dalla libreria di consinstenza, penalizzando più fortemente l'introduzione di gap in regioni altamente supportate e consentendo maggiore flessibilità in regioni debolmente vincolate. Sebbene tali raffinamenti non siano strettamente necessari per definire il metodo T-Coffee, essi contribuiscono in modo significativo alla qualità biologica all'allineamento finale.

3.3 Confronto tra T-Coffee e allineamento progressivo classico (Clustal)

L'algoritmo T-Coffee si colloca concettualmente all'interno della famiglia dei metodi di allineamento progressivo, condividendo con strumenti come ClustalW la stessa architettura generale basata su un albero guida e sull'allineamento iterativo di sequenze e profili. Tuttavia, la differenza fondamentale tra i due approcci risiede nella funzione obiettivo utilizzata durante la fase di allineamento progressivo.

Nei metodi progressivi classici, come ClustalW, il punteggio per allineare due residui o due colonne è calcolato tramite una funzione *Sum-of-Pairs* basata su una matrice di sostituzione (ad esempio BLOSUM o PAM) e su penalità di gap. Tale funzione riflette una similarità locale tra residui, valutata indipendentemente per ciascuna coppia di sequenze. Le decisioni prese nelle prime fasi dell'allineamento sono irrevocabili e gli errori introdotti precocemente vengono propagati lungo tutto l'MSA.

Al contrario, T-Coffee sostituisce la matrice di sostituzione con una funzione di scoring basata su una libreria di vincoli residuo-residuo estesa per consistenza. In questo schema, la programmazione dinamica non valuta più la sola similarità locale tra residui, ma il grado di supporto globale che una corrispondenza riceve dall'intero insieme delle sequenze. Un match può quindi essere favorito anche in presenza di una bassa similarità diretta, purchè sia coerente con gli allineamenti che coinvolgono sequenze intermedie.

Dal punto di vista algoritmico, è importante sottolineare che T-Coffee non introduce una nuova forma di programmazione dinamica: la struttura del DP rimane sostanzialmente identica a quella utilizzata da Clustal. La differenza risiede esclusivamente nella definizione della funzione di scoring, che trasforma un approccio puramente *greedy* in una procedura più robusta, capace di attenuare la propagazione di errori iniziali.

In sintesi, mentre Clustal privilegia l'efficienza computazionale e una valutazione locale delle corrispondenze, T-Coffee sacrifica parte della scalabilità per ottenere un allineamento biologicamente più affidabile, soprattutto nel caso di famiglie di sequenze distanti o caratterizzate da basse percentuali d'identità.

	ClustalW	T-Coffee
Architettura	Progressiva	Progressiva
Funzione di scoring	Matrice + SoP	Libreria di consistenza
Uso di sequenze intermedie	No	Sì
Robustezza agli errori iniziali	Bassa	Alta
Costo computazionale	Basso	Elevato

3.4 Analisi della Complessità Computazionale

L'accuratezza superiore di T-Coffee comporta un costo computazionale significativo. Mentre un allineamento progressivo standard ha una complessità temporale di $O(N^2L^2)$, T-Coffee introduce un fattore cubico rispetto al numero delle sequenze a causa della fase di estensione della

libreria. La complessità per la fase di estensione, che richiede il confronto di triplette di sequenze, è approssimabile a:

$$O(N^3L)$$

Dove N è il numero di sequenze e L la loro lunghezza. Questo rende T-Coffee computazionalmente oneroso per dataset contenenti un elevato numero di sequenze (es. $N > 100 - 200$), limitandone l'uso a set di dati di dimensioni moderate dove la precisione è prioritaria rispetto alla velocità.

3.5 Vantaggi e Limitazioni

Vantaggi:

- Accuratezza: T-Coffee è significativamente più accurato di ClustalW e altri metodi puramente progressivi, specialmente quando le sequenze hanno una bassa percentuale di identità (< 30)
- Flessibilità: Può integrare informazioni eterogenee (es. dati strutturali PDB o profili pre-esistenti) nella libreria iniziale senza modificare l'algoritmo di base.

Limitazioni:

- Scalabilità: Il costo $O(N^3)$ e l'elevata richiesta di memoria RAM per memorizzare la libreria lo rendono inadatto per l'allineamento di interi genomi o database massivi. Per ovviare a ciò, sono state sviluppate varianti come M-Coffee (meta-aligner) o versioni parallelizzate.
- Elevato consumo di memoria, necessario per memorizzare la libreria estesa

3.6 Integrazione di allineamenti locali e globali nella libreria

Uno degli aspetti distintivi di T-Coffee è la capacità di integrare informazioni eterogenee all'interno di una singola libreria di vincoli. A differenza dei metodi progressivi classici, che si basano prevalentemente su allineamenti globali, T-Coffee popola la libreria primaria combinando allineamenti globali e locali per ogni coppia di sequenze.

Gli allineamenti globali (tipicamente ottenuti con ClustalW) forniscono una copertura completa della sequenza e sono appropriati quando l'omologia è estesa lungo tutta la lunghezza. Gli allineamenti locali (ottenuti con LALIGN/Smith–Waterman) permettono invece di individuare domini conservati o motivi funzionali anche quando il contesto globale è altamente divergente o quando le sequenze hanno lunghezze differenti.

La funzione obiettivo basata sulla consistenza agisce come un meccanismo di integrazione e selezione: vincoli supportati coerentemente sia da allineamenti locali sia globali ricevono un peso elevato nella libreria, mentre vincoli contraddittori o isolati vengono attenuati. Questo approccio consente di bilanciare copertura e sensibilità, rendendo T-Coffee particolarmente efficace nell'allineamento di proteine multidominio o di famiglie con bassa identità globale.

3.7 Valutazione dell'affidabilità: indice CORE

Un vantaggio dell'approccio consistency-based è la possibilità di stimare l'affidabilità *locale* dell'allineamento finale. T-Coffee introduce l'indice **CORE** (*Consistency of Overall Residue Evaluation*), che assegna a ciascun residuo (o colonna) un punteggio discreto (tipicamente su scala 0–9) proporzionale al supporto fornito dalla libreria estesa per la corrispondenza scelta nell'MSA. Valori elevati indicano regioni robustamente supportate dalla maggioranza delle sequenze, mentre valori bassi evidenziano tratti ambigui (ad esempio loop, inserti o regioni a bassa complessità).

Operativamente, l'indice CORE può essere utilizzato per (i) evidenziare visivamente le regioni affidabili nell'output, (ii) filtrare o mascherare porzioni di allineamento a bassa confidenza in analisi downstream (filogenesi, ricerca di siti conservati), e (iii) confrontare la stabilità di alternative di allineamento in regioni critiche.

3.8 Conclusioni sul Metodo

In conclusione, T-Coffee rappresenta l'evoluzione qualitativa dell'allineamento progressivo. Pur mantenendo l'euristica gerarchica necessaria per trattare allineamenti multipli, risolve parzialmente il problema del posizionamento errato dei gap attraverso un pre-processing "consapevole" dell'intero dataset. Esso dimostra che l'informazione contenuta in sequenze intermedie è fondamentale per allineare correttamente sequenze distanti, spostando il focus dall'ottimizzazione locale (pairwise) a quella globale (consistency).

Mentre l'allineamento progressivo fornisce l'impalcatura algoritmica necessaria per gestire N sequenze, T-Coffee ne raffina il contenuto informativo. Sostituendo le matrici statiche con una valutazione basata sulla consistenza globale, T-Coffee mitiga la natura irreversibile dell'approccio greedy, trasformando l'allineamento progressivo da una semplice euristica veloce a uno strumento ad alta precisione biologica.

3.9 Integrazione di Allineamenti Locali e Globali nella Libreria

La potenza dell'algoritmo T-Coffee risiede nella sua capacità di combinare informazioni eterogenee all'interno della stessa libreria primaria. Mentre i metodi progressivi standard tendono a favorire l'allineamento globale (coprendo l'intera lunghezza della sequenza), T-Coffee popola la sua libreria utilizzando due distinti algoritmi per ogni coppia di sequenze:

1. **Metodo Globale (ClustalW):** Utile per allineare sequenze di lunghezza simile e altamente omologhe.

2. **Metodo Locale (LALIGN):** Fondamentale per identificare domini conservati o motivi funzionali anche quando le sequenze circostanti sono molto divergenti o di lunghezza differente.

La funzione obiettivo basata sulla consistenza agisce quindi come un arbitro: se un residuo x è allineato a y sia dall'algoritmo locale che da quello globale, il peso nella libreria sarà alto. Se i due metodi sono in disaccordo, il peso sarà inferiore, riflettendo l'incertezza posizionale. Questo approccio ibrido rende T-Coffee particolarmente robusto nell'analisi di proteine multidominio.

3.10 Evoluzione della Piattaforma T-Coffee: dal 2000 al 2025

Dalla sua pubblicazione originale nel 2000, T-Coffee si è evoluto da singolo algoritmo di allineamento a piattaforma di integrazione versatile (spesso definita *meta-aligner*). Lo sviluppo del software ha seguito due direttrici principali: nel primo decennio l'obiettivo primario è stato il miglioramento dell'accuratezza attraverso l'integrazione di dati eterogenei; nel decennio successivo, con l'avvento delle tecnologie di sequenziamento massivo (NGS), il focus si è spostato sulla scalabilità computazionale.

3.10.1 L'Era dell'Accuratezza e dell'Integrazione (2000-2010)

In questa fase, sono state sviluppate varianti specializzate per risolvere limitazioni specifiche dell'allineamento basato sulla sola sequenza primaria.

- **M-Coffee (Meta-Coffee):** Introdotto per mitigare gli errori sistematici dei singoli algoritmi. M-Coffee costruisce la libreria di consistenza combinando gli output di molteplici software di allineamento terzi (es. MUSCLE, MAFFT, ProbCons). Il risultato finale è un allineamento di "consenso" che, statisticamente, supera in accuratezza ogni singolo metodo individuale.

- **Espresso (3D-Coffee):** Rappresenta un salto di qualità nell'analisi di sequenze distanti (<20% di identità). Questa variante interroga il database PDB per verificare l'esistenza di strutture cristallografiche associate alle sequenze in input. Se presenti, la libreria viene popolata utilizzando allineatori strutturali (come SAP o TM-align) invece di allineatori di sequenza, sfruttando il principio che la struttura terziaria è evolutivamente più conservata della sequenza primaria.
- **R-Coffee:** Una variante specifica per l'allineamento di molecole di RNA non codificante. R-Coffee integra nella libreria informazioni sulla struttura secondaria (predetta o sperimentale), penalizzando l'inserimento di gap che romperebbero i legami idrogeno nelle eliche (stem) dell'RNA, garantendo così la conservazione della funzionalità biologica nel modello finale.
- **PSI-Coffee:** Ispirato alla logica di PSI-BLAST, questo metodo è progettato per sequenze con scarsa omologia rilevabile. Per ogni sequenza di input, l'algoritmo genera un profilo di omologia cercando sequenze simili nei database pubblici; l'allineamento viene poi guidato da questi profili arricchiti piuttosto che dalle singole sequenze, aumentando notevolmente la sensibilità.

3.10.2 L'Era della Scalabilità e dell'Integrazione Strutturale Predittiva (2015-2025)

Per rispondere al limite della complessità cubica $O(N^3)$ e all'esplosione dei dati genomici, gli sviluppi recenti hanno rivoluzionato l'architettura interna del software.

- **Regressive T-Coffee:** Questa innovazione algoritmica abbandona l'approccio progressivo monolitico per un metodo "divide et impera". L'algoritmo frammenta il dataset iniziale (anche di 100.000+ sequenze) in sottogruppi più piccoli, li allinea separatamente con l'alta precisione del metodo classico, e infine

ricombina i profili risultanti. Ciò riduce la complessità computazionale a livelli quasi lineari $O(N \log N)$, rendendo T-Coffee competitivo con MAFFT e MUSCLE su grandi dataset senza sacrificare eccessivamente l'accuratezza.

- **Integrazione con AlphaFold (Fold-Coffee):** Con la rivoluzione del Deep Learning nella biologia strutturale, le versioni più recenti hanno integrato il supporto per modelli predittivi. T-Coffee può ora utilizzare le strutture ad alta confidenza generate da AlphaFold2 per guidare l'allineamento tramite la modalità Expresso, estendendo i benefici dell'allineamento strutturale anche a proteine che non sono mai state cristallizzate sperimentalmente.
- **Riproducibilità e Cloud Computing:** Parallelamente all'algoritmo, l'ecosistema si è evoluto con l'introduzione di framework come *Nextflow*. T-Coffee è stato ottimizzato per l'esecuzione in container (Docker/Singularity), facilitandone l'impiego in pipeline bioinformatiche distribuite su cloud, un requisito essenziale per la moderna genomica su larga scala.

A Implementazione: scheletro Python per T-Coffee

In questa appendice viene presentata un'implementazione didattica ed eseguibile in Python dell'algoritmo T-Coffee, coerente con l'architettura descritta nel Capitolo 5. Pur non avendo l'obiettivo di competere con implementazioni ottimizzate o industriali, il codice realizza una pipeline completa e funzionante, comprendente la costruzione della libreria primaria, la sua estensione per consistenza, lo scoring basato sulla libreria e l'allineamento progressivo con traceback, al fine di rendere esplicita la traduzione operativa dei concetti teorici discussi nel testo.

A.1 Struttura generale

```
1 from __future__ import annotations
2 from dataclasses import dataclass
3 from typing import Dict, Tuple, List, Optional
4
5 # -----
6 # Data structures
7 # -----
8
9 Resid = Tuple[int, int]           # residui(seq_id, pos0) 0-
   based
10 LibKey = Tuple[int, int, int, int] # coppia di residui allineati
   (i, a, j, b) with i<j
11 Library = Dict[LibKey, float]     # dizionario: chiave= coppia
   di residui , valore= peso/confidenzaa
12
13
14 #evita duplicati
15 def norm_key(i: int, a: int, j: int, b: int) -> LibKey:
16     """Ensure i<j ordering for library keys."""
17     if i < j:
18         return (i, a, j, b)
19     return (j, b, i, a)
20
21
22 #righe di un MSA
23 @dataclass
24 class AlnRow:
25     seq_id: int
```

```

26     aligned: str                                #sequenza allineata con gap
27     col_to_pos: List[Optional[int]]             # aligned column -> original
                                                posizione (None if gap)
28
29 Profile = List[AlnRow] #lista di righe
30
31 # -----
32 # Needleman Wunsch (global, gap lineare) + traceback
33 # -----
34 def needleman_wunsch_global(
35     x: str,
36     y: str,
37     match: int = 1,
38     mismatch: int = -1,
39     gap: int = -1,
40 ) -> Tuple[str, str]:
41     m, n = len(x), len(y)
42     S = [[0] * (n + 1) for _ in range(m + 1)]
43     TB = [[""] * (n + 1) for _ in range(m + 1)]
44
45     for i in range(1, m + 1):
46         S[i][0] = S[i - 1][0] + gap
47         TB[i][0] = "U"
48     for j in range(1, n + 1):
49         S[0][j] = S[0][j - 1] + gap
50         TB[0][j] = "L"
51
52     for i in range(1, m + 1):
53         xi = x[i - 1]
54         for j in range(1, n + 1):
55             yj = y[j - 1]
56             s_sub = match if xi == yj else mismatch
57
58             diag = S[i - 1][j - 1] + s_sub
59             up = S[i - 1][j] + gap
60             left = S[i][j - 1] + gap
61
62             best = max(diag, up, left)
63             S[i][j] = best
64
65             if best == diag:
66                 TB[i][j] = "D"
67             elif best == up:
68                 TB[i][j] = "U"
69             else:
70                 TB[i][j] = "L"
71
72     ai: List[str] = []
73     aj: List[str] = []

```

```

74     i, j = m, n
75     while i > 0 or j > 0:
76         if i == 0:
77             step = "L"
78         elif j == 0:
79             step = "U"
80         else:
81             step = TB[i][j]
82
83         if step == "D":
84             ai.append(x[i - 1])
85             aj.append(y[j - 1])
86             i -= 1
87             j -= 1
88         elif step == "U":
89             ai.append(x[i - 1])
90             aj.append("-")
91             i -= 1
92         else: # "L"
93             ai.append("-")
94             aj.append(y[j - 1])
95             j -= 1
96
97     ai.reverse()
98     aj.reverse()
99     return "".join(ai), "".join(aj)
100
101 def percent_identity(a: str, b: str) -> float:
102     #conta match ignorando colonne con gap
103     matches = 0
104     aligned = 0
105     for x, y in zip(a, b):
106         if x == "-" or y == "-":
107             continue
108         aligned += 1
109         if x == y:
110             matches += 1
111     return (matches / aligned) if aligned else 0.0
112
113 # -----
114 # 1 Primary library (toy)
115 # -----
116 #per ogni coppia(i,j) fai la NW globale --> calcoli il
117 #percentity_identity e scorre la colonna
118 #fin quando hai una coppia di residui (non gap) e aggiorni la
119 #libreria
120 def build_primary_library(
121     seqs: List[str],
122     nw_match: int = 1,

```

```

121     nw_mismatch: int = -1,
122     nw_gap: int = -1,
123 ) -> Library:
124     lib: Library = {}
125     for i in range(len(seqs)):
126         for j in range(i + 1, len(seqs)):
127             ai, aj = needleman_wunsch_global(
128                 seqs[i], seqs[j], match=nw_match, mismatch=
nw_mismatch, gap=nw_gap
129             )
130             w_source = percent_identity(ai, aj)
131
132             pi = -1
133             pj = -1
134             for c1, c2 in zip(ai, aj):
135                 if c1 != "-":
136                     pi += 1
137                 if c2 != "-":
138                     pj += 1
139                 if c1 != "-" and c2 != "-":
140                     key = norm_key(i, pi, j, pj)
141                     lib[key] = lib.get(key, 0.0) + w_source
142     return lib
143
144 # -----
145 # 2 Library extension (consistency)
146 # -----
147
148 #costruisce una liswta di vicinato neigh
149 #per ogni coppia (r1 r2, peso w) aggiunge r2 nei vicini di r1 e
viceversa
150 #poi per ogni cammino a due passi r -> mid -> t aggiunge:
151 #min(w_rm, w_mt) oppure w_rm*w_mt a seconda di mode
152 def extend_library(primary: Library, mode: str = "min") -> Library:
153     ext = dict(primary)
154     neigh: Dict[Resid, List[Tuple[Resid, float]]] = {}
155
156     for (i, a, j, b), w in primary.items():
157         r1: Resid = (i, a)
158         r2: Resid = (j, b)
159         neigh.setdefault(r1, []).append((r2, w))
160         neigh.setdefault(r2, []).append((r1, w))
161
162     def f(x: float, y: float) -> float:
163         if mode == "min":
164             return min(x, y)
165         if mode == "prod":
166             return x * y
167         raise ValueError("mode must be 'min' or 'prod'")

```

```

168
169     for r, edges in neigh.items():
170         for mid, w_rm in edges:
171             for t, w_mt in neigh.get(mid, []):
172                 if t == r:
173                     continue
174                 (i, a) = r
175                 (j, b) = t
176                 if i == j:
177                     continue
178                 key = norm_key(i, a, j, b)
179                 ext[key] = ext.get(key, 0.0) + f(w_rm, w_mt)
180
181     return ext
182
183 # -----
184 # 3 Library-based column scoring
185 # -----
186 def library_column_score(
187     colA: List[Optional[Resid]],
188     colB: List[Optional[Resid]],
189     lib_ext: Library
190 ) -> float:
191     score = 0.0
192     for ra in colA:
193         if ra is None:
194             continue
195         for rb in colB:
196             if rb is None:
197                 continue
198             (i, a) = ra
199             (j, b) = rb
200             if i == j:
201                 continue
202             score += lib_ext.get(norm_key(i, a, j, b), 0.0)
203     return score
204
205 # -----
206 # 4 Align profiles with DP + traceback (gap affine)
207 # -----
208
209 #trasforma righe in colonne
210 def profile_to_columns(profile: Profile) -> List[List[Optional[Resid]
211 ]]]:
212     n_cols = len(profile[0].aligned)
213     cols: List[List[Optional[Resid]]] = []
214     for c in range(n_cols):
215         col: List[Optional[Resid]] = []
216         for row in profile:

```



```

216         pos = row.col_to_pos[c]
217         col.append((row.seq_id, pos) if pos is not None else None
218     )
219     cols.append(col)
220     return cols
221
222 def align_profiles_with_library(
223     profA: Profile,
224     profB: Profile,
225     lib_ext: Library,
226     gap_open: float = -2.0,
227     gap_ext: float = -0.5,
228 ) -> Profile:
229     colsA = profile_to_columns(profA)
230     colsB = profile_to_columns(profB)
231     m, n = len(colsA), len(colsB)
232
233     NEG = -1e18
234     M = [[NEG] * (n + 1) for _ in range(m + 1)]
235     Ix = [[NEG] * (n + 1) for _ in range(m + 1)]
236     Iy = [[NEG] * (n + 1) for _ in range(m + 1)]
237
238     tbM = [[("?", 0, 0)] * (n + 1) for _ in range(m + 1)]
239     tbIx = [[("?", 0, 0)] * (n + 1) for _ in range(m + 1)]
240     tbIy = [[("?", 0, 0)] * (n + 1) for _ in range(m + 1)]
241
242     M[0][0] = 0.0
243
244     for i in range(1, m + 1):
245         Ix[i][0] = gap_open + (i - 1) * gap_ext
246         tbIx[i][0] = ("Ix", i - 1, 0)
247     for j in range(1, n + 1):
248         Iy[0][j] = gap_open + (j - 1) * gap_ext
249         tbIy[0][j] = ("Iy", 0, j - 1)
250
251     for i in range(1, m + 1):
252         for j in range(1, n + 1):
253             s = library_column_score(colsA[i - 1], colsB[j - 1],
254                                     lib_ext)
255
256             candidates = [
257                 (M[i - 1][j - 1] + s, ("M", i - 1, j - 1)),
258                 (Ix[i - 1][j - 1] + s, ("Ix", i - 1, j - 1)),
259                 (Iy[i - 1][j - 1] + s, ("Iy", i - 1, j - 1)),
260             ]
261             M[i][j], tbM[i][j] = max(candidates, key=lambda x: x[0])
262
263     cand_ix = [
264         (M[i - 1][j] + gap_open, ("M", i - 1, j)),

```

```

263         (Ix[i - 1][j] + gap_ext, ("Ix", i - 1, j)),
264     ]
265     Ix[i][j], tbIx[i][j] = max(cand_ix, key=lambda x: x[0])
266
267     cand_iy = [
268         (M[i][j - 1] + gap_open, ("M", i, j - 1)),
269         (Iy[i][j - 1] + gap_ext, ("Iy", i, j - 1)),
270     ]
271     Iy[i][j], tbIy[i][j] = max(cand_iy, key=lambda x: x[0])
272
273     end_candidates = [(M[m][n], "M"), (Ix[m][n], "Ix"), (Iy[m][n], "
Iy")]
274     _, state = max(end_candidates, key=lambda x: x[0])
275
276     ops: List[str] = []
277     i, j = m, n
278     while i > 0 or j > 0:
279         if state == "M":
280             prev_state, pi, pj = tbM[i][j]
281             ops.append("D")
282         elif state == "Ix":
283             prev_state, pi, pj = tbIx[i][j]
284             ops.append("A")
285         else:
286             prev_state, pi, pj = tbIy[i][j]
287             ops.append("B")
288         state, i, j = prev_state, pi, pj
289
290     ops.reverse()
291
292     merged: List[AlnRow] = [AlnRow(r.seq_id, "", []) for r in (profA
+ profB)]
293
294     def append_from_profile(rows: List[AlnRow], src: Profile, col_idx
: Optional[int]):
295         offset = 0 if src is profA else len(profA)
296         for r_i, src_row in enumerate(src):
297             target = rows[offset + r_i]
298             if col_idx is None:
299                 target.aligned += "-"
300                 target.col_to_pos.append(None)
301             else:
302                 target.aligned += src_row.aligned[col_idx]
303                 target.col_to_pos.append(src_row.col_to_pos[col_idx])
304
305     a_col = 0
306     b_col = 0
307     for op in ops:
308         if op == "D":

```

```

309         append_from_profile(merged, profA, a_col)
310         append_from_profile(merged, profB, b_col)
311         a_col += 1
312         b_col += 1
313     elif op == "A":
314         append_from_profile(merged, profA, a_col)
315         append_from_profile(merged, profB, None)
316         a_col += 1
317     else:
318         append_from_profile(merged, profA, None)
319         append_from_profile(merged, profB, b_col)
320         b_col += 1
321
322     return merged
323
324     # -----
325     # Guide tree (UPGMA) + progressive
326     # -----
327 def make_singleton_profiles(seqs: List[str]) -> Dict[int, Profile]:
328     profs: Dict[int, Profile] = {}
329     for sid, s in enumerate(seqs):
330         profs[sid] = [AlnRow(seq_id=sid, aligned=s, col_to_pos=list(
331             range(len(s))))]
332     return profs
333
334 def pairwise_distance_matrix(
335     seqs: List[str],
336     match: int = 1,
337     mismatch: int = -1,
338     gap: int = -1,
339 ) -> List[List[float]]:
340     n = len(seqs)
341     D = [[0.0] * n for _ in range(n)]
342     for i in range(n):
343         for j in range(i + 1, n):
344             ai, aj = needleman_wunsch_global(seqs[i], seqs[j], match,
345                 mismatch, gap)
346             pid = percent_identity(ai, aj)
347             dist = 1.0 - pid
348             D[i][j] = D[j][i] = dist
349     return D
350
351 def upgma_merges(D: List[List[float]]) -> Tuple[List[Tuple[int, int,
352     int]], int]:
353     """
354     Returns:
355         merges: list of (ca, cb, newc)
356         root_id: final cluster id
357     """

```

```

355     n = len(D)
356     active = {i for i in range(n)}
357     members: Dict[int, List[int]] = {i: [i] for i in range(n)}
358     next_id = n
359     merges: List[Tuple[int, int, int]] = []
360
361     def avg_dist(ca: int, cb: int) -> float:
362         A = members[ca]
363         B = members[cb]
364         tot = 0.0
365         cnt = 0
366         for i in A:
367             for j in B:
368                 tot += D[i][j]
369                 cnt += 1
370         return tot / cnt if cnt else 0.0
371
372     last_new = None
373     while len(active) > 1:
374         best_pair = None
375         best_val = 1e18
376         act_list = sorted(active)
377         for ia in range(len(act_list)):
378             for ib in range(ia + 1, len(act_list)):
379                 ca = act_list[ia]
380                 cb = act_list[ib]
381                 v = avg_dist(ca, cb)
382                 if v < best_val:
383                     best_val = v
384                     best_pair = (ca, cb)
385
386     assert best_pair is not None
387     ca, cb = best_pair
388
389     newc = next_id
390     next_id += 1
391
392     merges.append((ca, cb, newc))
393     members[newc] = members[ca] + members[cb]
394     active.remove(ca)
395     active.remove(cb)
396     active.add(newc)
397     last_new = newc
398
399     assert last_new is not None
400     return merges, last_new
401
402 def progressive_tcoffee_toy(
403     seqs: List[str],

```

```

404     gap_open: float = -2.0,
405     gap_ext: float = -0.5,
406     nw_match: int = 1,
407     nw_mismatch: int = -1,
408     nw_gap: int = -1,
409     ext_mode: str = "min",
410 ) -> Profile:
411     n = len(seqs)
412     if n == 0:
413         return []
414     if n == 1:
415         return [AlnRow(0, seqs[0], list(range(len(seqs[0]))))]
416
417     primary = build_primary_library(seqs, nw_match, nw_mismatch,
418     nw_gap)
419     print("Primary library links:", len(primary))
420     lib_ext = extend_library(primary, mode=ext_mode)
421     print("Extended library links:", len(lib_ext))
422
423     D = pairwise_distance_matrix(seqs, nw_match, nw_mismatch, nw_gap)
424     merges, root_id = upgma_merges(D)
425
426     profiles = make_singleton_profiles(seqs)
427
428     for ca, cb, newc in merges:
429         merged = align_profiles_with_library(
430             profiles[ca], profiles[cb], lib_ext,
431             gap_open=gap_open, gap_ext=gap_ext
432         )
433         profiles[newc] = merged
434
435     final_profile = profiles[root_id]
436     final_profile.sort(key=lambda row: row.seq_id)
437     return final_profile
438
439 def print_profile(profile: Profile, names: Optional[List[str]] = None
440 ) -> None:
441     for row in profile:
442         label = f"S{row.seq_id}"
443         if names and row.seq_id < len(names):
444             label = names[row.seq_id]
445         print(f">{label}\n{row.aligned}")
446
447 def run_demo():
448     seqs = [
449         "ACGTGCA",
450         "ACGTCGA",
451         "ACGAGCA",
452         "ACGTGGA",

```

```

451     ]
452     names = [f"Seq{i+1}" for i in range(len(seqs))]
453
454     # --- evidence: build libraries ---
455     primary = build_primary_library(seqs)
456     ext = extend_library(primary)
457
458     print("Primary library links:", len(primary))
459     print("Extended library links:", len(ext))
460
461     # --- run full T-Coffee toy pipeline ---
462     msa = progressive_tcoffee_toy(seqs)
463
464     print("\nMSA finale T-Coffee toy:")
465     print_profile(msa, names=names)
466
467     # --- automatic checks (proof that it works) ---
468     assert len(primary) > 0
469     assert len(ext) >= len(primary)
470     assert all(len(r.aligned) == len(msa[0].aligned) for r in msa)
471
472     print("\nOK: pipeline completa e allineamento consistente.")
473
474 if __name__ == "__main__":
475     run_demo()

```

Listing 2: Scheletro Python per T-Coffee (versione didattica)

Bibliografia

- [1] Da-Fei Feng e Russell F. Doolittle. “Progressive sequence alignment as a prerequisite to correct phylogenetic trees”. In: *Journal of Molecular Evolution* 25.4 (1987), pp. 351–360.
- [2] Julie D. Thompson, Desmond G. Higgins e Toby J. Gibson. “CLUSTAL W: improving the sensitivity of progressive multiple sequence alignment through sequence weighting, position-specific gap penalties and weight matrix choice”. In: *Nucleic Acids Research* 22.22 (1994), pp. 4673–4680.
- [3] Cedric Notredame, Desmond G. Higgins e Jaap Heringa. “T-Coffee: A novel method for fast and accurate multiple sequence alignment”. In: *Journal of Molecular Biology* 302.1 (2000), pp. 205–217.
- [4] Kazutaka Katoh et al. “MAFFT: a novel method for rapid multiple sequence alignment based on fast Fourier transform”. In: *Nucleic Acids Research* 30.14 (2002), pp. 3059–3066.
- [5] Robert C. Edgar. “MUSCLE: multiple sequence alignment with high accuracy and high throughput”. In: *Nucleic Acids Research* 32.5 (2004), pp. 1792–1797.
- [6] Julie D. Thompson, Frederic Plewniak e Olivier Poch. “A comprehensive comparison of multiple sequence alignment programs”. In: *Nucleic Acids Research* 27.13 (1999), pp. 2682–2690.
- [7] Julie D. Thompson et al. “BALiBASE 3.0: latest developments of the multiple sequence alignment benchmark”. In: *Proteins* 61.1 (2005), pp. 127–136.
- [8] Anders Krogh et al. “Hidden Markov models in computational biology”. In: *Journal of Molecular Biology* 235.5 (1994), pp. 1501–1531.
- [9] Cedric Notredame. “Recent progress in multiple sequence alignment: a survey”. In: *Pharmacogenomics* 3.1 (2002), pp. 131–144.

- [10] Naruya Saitou e Masatoshi Nei. “The neighbor-joining method: a new method for reconstructing phylogenetic trees”. In: *Molecular Biology and Evolution* 4.4 (1987), pp. 406–425.
- [11] Saul B. Needleman e Christian D. Wunsch. “A general method applicable to the search for similarities in the amino acid sequence of two proteins”. In: *Journal of Molecular Biology* 48.3 (1970), pp. 443–453.
- [12] Richard Durbin et al. *Biological sequence analysis: probabilistic models of proteins and nucleic acids*. Cambridge University Press, 1998.
- [13] David W. Mount. *Bioinformatics: sequence and genome analysis*. Cold Spring Harbor Laboratory Press, 2004.
- [14] Marketa J. Zvelebil e Jeremy O. Baum. *Understanding Bioinformatics*. Garland Science, 2008.
- [15] Pavel Pevzner e Phillip Compeau. *Bioinformatics Algorithms: An Active Learning Approach*. Active Learning Publishers, 2014.